МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №2
по дисциплине
«Основы машинного обучения»
«Многослойные нейронные сети и обратное распространение ошибки»

Выполнил студент гр. ИВТб-2303-06-00 _____ /Перевозчиков Д.А./

Проверил _____ /Пащенко Д. Э./

Киров
2026

1 Цель работы

Изучить устройство и обучение многослойной нейронной сети, освоить метод обратного распространения ошибки (BackProp) и получить практические навыки постановки воспроизводимого эксперимента с нейросетевой моделью. В работе вручную реализуются прямой и обратный проход двухслойной сети в NumPy с проверкой градиентов конечными разностями и обучением на задаче XOR; затем в PyTorch обучается MLP на наборе mfeat-karhunen, сравниваются оптимизаторы SGD, Momentum и Adam, исследуются Dropout и L2-регуляризация, а результат сопоставляется с линейным baseline.

2 Вариант и данные

Особенности набора: Все 64 признака являются числовыми (коэффициенты Кархунена-Лоэва, описывающие форму объектов). Пропуски отсутствуют. Датасет сбалансирован (ровно 200 объектов на каждый из 10 классов), поэтому применять методы борьбы с дисбалансом не требуется, однако macro-усреднение метрик для объективной оценки приходится применить.

Параметр	**Значение**
RANDOM_STATE	1004
Разбиение	`test_size=0.2`, `stratify=y`, 1600 объектов в Train и 400 в Test
Кросс-валидация	`StratifiedKFold(n_splits=5, shuffle=True)`
Метрики	`macro-F1` (так как классов 10, и они равнозначны). Accuracy, macro-Precision, macro-Recall, ROC-AUC, cross-entropy, нормированная confusion matrix
Правила preprocessing	Так как пропусков нет, импутация данных не требуется. Все числовые признаки масштабируются с помощью `StandardScaler` для корректной работы градиентного спуска

Таблица 1: Параметры экспериментального протокола

Классы распределены строго поровну (рисунок 1), поэтому методы борьбы с дисбалансом не требуются. На рисунке 2 показаны два графика признаков: распределение `att1` по классам и расположение объектов в плоскости (`att1`, `att2`). Признак `att1` хорошо разделяет классы 1, 7 и 8, тогда как классы 4 и 6 имеют широкий разброс, а в плоскости (`att1`, `att2`) классы 2, 3, 4 и 9 заметно перемешаны.

3 Краткие теоретические сведения

Для мини-пакета X полносвязный слой записывается как $Z = XW + b$, $H = \varphi(Z)$. Без нелинейной активации φ композиция слоёв эквивалентна одному линейному преобразованию, поэтому задача XOR не решается одним линейным нейроном. Для сети с одним скрытым слоем и ReLU прямой

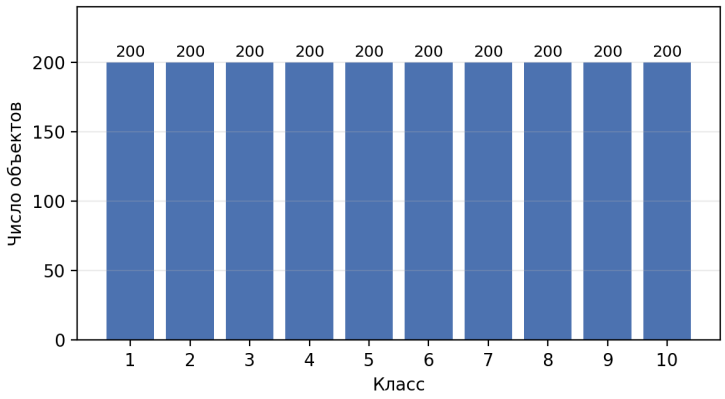


Рис. 1: Распределение объектов по классам

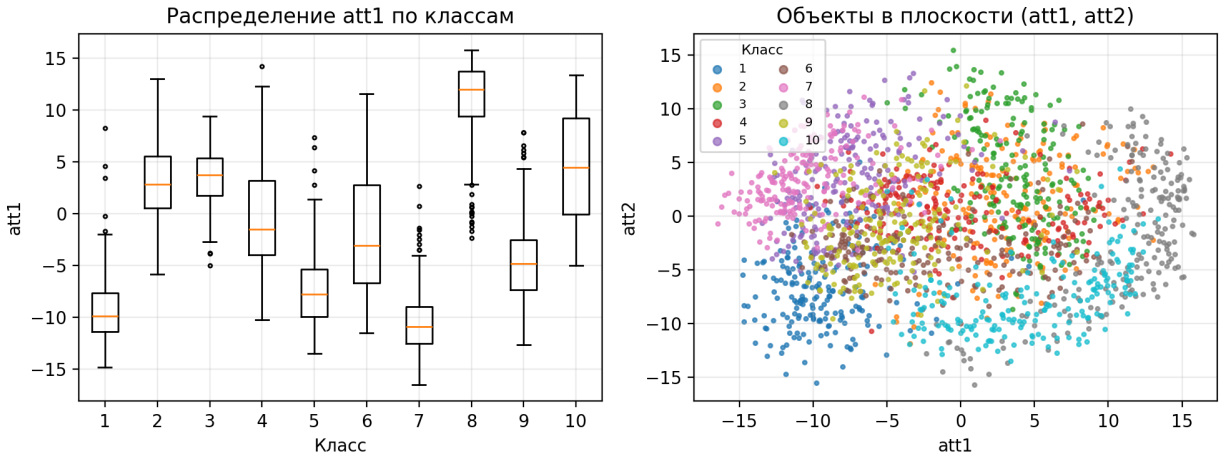


Рис. 2: Признак `att1` по классам и объекты в плоскости (`att1`, `att2`)

проход и функция потерь имеют вид

$$Z^1 = XW^1 + b^1, \quad H = \mathrm{ReLU}(Z^1), \quad Z^2 = HW^2 + b^2, \quad P = \mathrm{softmax}(Z^2),$$
$$L = -\frac{1}{m}\sum_i \ln P_{i,y_i} + \frac{\lambda}{2}\left(\|W^1\|^2 + \|W^2\|^2\right).$$

Обратное распространение применяет правило цепочки справа налево. Для связки Softmax и кросс-энтропии $dZ^2 = (P - Y)/m$, далее

$$dW^2 = H^\top dZ^2 + \lambda W^2, \quad db^2 = \sum dZ^2, \quad dH = dZ^2 (W^2)^\top,$$
$$dZ^1 = dH \odot \mathbf{1}[Z^1 > 0], \quad dW^1 = X^\top dZ^1 + \lambda W^1, \quad db^1 = \sum dZ^1.$$

На прямом проходе сохраняются X, Z^1, H и P. Градиент проверяется центральной конечной разностью $g_{num} \approx [L(\theta + \varepsilon) - L(\theta - \varepsilon)]/(2\varepsilon)$. Оптимизаторы: SGD — $\theta \leftarrow \theta - \eta g$; Momentum — $v \leftarrow \beta v + g$, $\theta \leftarrow \theta - \eta v$;

Adam использует экспоненциальные средние градиента и его квадрата. L2-регуляризация ограничивает рост весов, Dropout зануляет часть активаций только в режиме обучения, ранняя остановка возвращает параметры с лучшей validation loss.

4 Ход работы и результаты

4.1 Линейный baseline

`LogisticRegression` обучена заново на Train текущего разбиения с той же стандартизацией, что и у MLP. На validation получены **Accuracy = 0.9325** и **macro-F1 = 0.9327**. Это согласуется с результатами лабораторной №1 (CV macro-F1 = 0.9443).

Гипотеза: нелинейная модель улучшит качество, так как классы частично пересекаются (рисунок 2) и линейная граница не может их полностью разделить; ограничивающий фактор — небольшой объём Train.

4.2 Ручной BackProp, gradient check и XOR

В NumPy реализована сеть с одним скрытым слоем: He-инициализация W^1, нулевые смещения, устойчивый Softmax (сдвиг на максимальный логит), функции прямого прохода, кросс-энтропии и обратного прохода. Ключевой фрагмент обратного прохода приведён в листинге 1.

```python
def backward_numpy(cache, y, parameters, l2=0.0):
    X, Z1, H, P = cache["X"], cache["Z1"], cache["H"], cache["P"]
    m = len(y)
    dZ2 = P.copy()
    dZ2[np.arange(m), y] -= 1.0
    dZ2 /= m                        # (P - Y) / m
    dW2 = H.T @ dZ2 + l2 * parameters["W2"]
    db2 = dZ2.sum(axis=0)
    dH = dZ2 @ parameters["W2"].T
    dZ1 = dH * (Z1 > 0)             # ReLU derivative
    dW1 = X.T @ dZ1 + l2 * parameters["W1"]
    db1 = dZ1.sum(axis=0)
    return {"W1": dW1, "b1": db1, "W2": dW2, "b2": db2}
```

Листинг 1: Обратный проход двухслойной сети (NumPy)

Проверка градиента. Проверка выполнена на малой сети (64–5–10) и 8 объектах Train, $\varepsilon = 10^{-5}$; случайно выбрано 12 компонент градиента (таблица 2). Максимальная относительная ошибка равна $4.81 \cdot 10^{-10}$: аналитический и численный градиенты совпадают. Элемент $b^1[3]$ выбран случайно дважды, поэтому различных компонент 11.

Параметр	Аналитический	Численный	Отн. ошибка
$W^2[0,0]$	0.284840	0.284840	$8.1 \cdot 10^{-12}$
$W^1[8,1]$	0.016103	0.016103	$4.6 \cdot 10^{-10}$
$W^1[8,4]$	0.047225	0.047225	$1.3 \cdot 10^{-10}$
$b^2[3]$	0.014238	0.014238	$4.8 \cdot 10^{-10}$
$b^1[3]$	0.018261	0.018261	$2.7 \cdot 10^{-10}$
$b^2[8]$	0.047566	0.047566	$2.4 \cdot 10^{-11}$
$b^1[0]$	0.106404	0.106404	$8.8 \cdot 10^{-11}$
$b^2[5]$	0.011965	0.011965	$1.2 \cdot 10^{-10}$
$b^1[1]$	-0.129908	-0.129908	$3.5 \cdot 10^{-11}$
$b^1[4]$	0.030470	0.030470	$1.5 \cdot 10^{-10}$
$b^1[3]$	0.018261	0.018261	$2.7 \cdot 10^{-10}$
$W^2[1,0]$	0.100170	0.100170	$3.8 \cdot 10^{-11}$

Таблица 2: Сравнение аналитических и численных градиентов

XOR. Сеть 2–8–2 обучалась полным градиентным спуском ($\eta = 0.10$,

L2 $= 10^{-4}$, 3000 эпох). Функция потерь снизилась до 0.0064 (рисунок 3), все четыре объекта классифицированы верно (таблица 3). Одной прямой точки XOR не разделить, но скрытый слой с ReLU строит новые признаки, в пространстве которых классы линейно разделимы; без нелинейности композиция слоёв осталась бы линейной моделью.

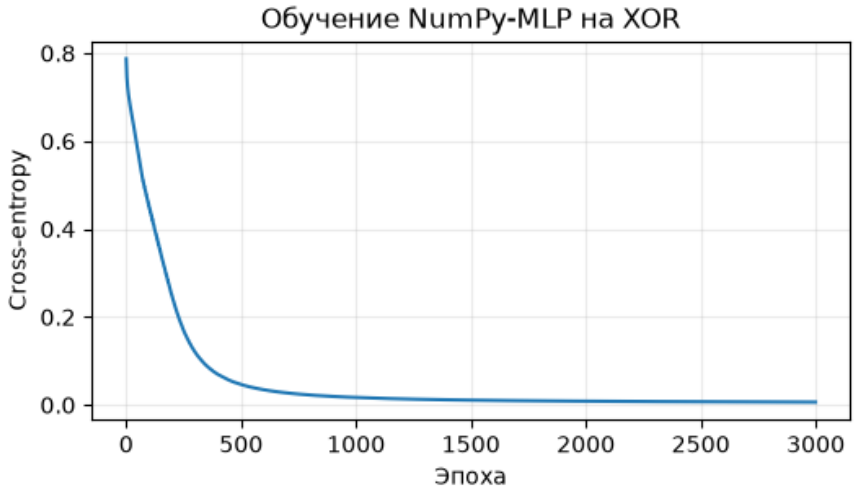


Рис. 3: Функция потерь NumPy-MLP при обучении на XOR

x_1	x_2	$P(y=1)$	Предсказание
0	0	0.010	0
0	1	0.998	1
1	0	0.999	1
1	1	0.001	0

Таблица 3: Результат NumPy-сети на XOR

4.3 MLP в PyTorch

Архитектура: `Linear(64, 64)` → `ReLU` → `Dropout(p)` → `Linear(64, 10)`. Скрытый слой инициализируется по He, выходной — по Xavier, смещения нулевые. Сеть возвращает логиты, поэтому `CrossEntropyLoss` получает их напрямую. Данные подаются через `TensorDataset` и `DataLoader` (batch 64 на обучении, 256 на оценке). Максимум 120 эпох, ранняя остановка с терпением 15 эпох по validation loss, по окончании восстанавливаются лучшие ве-

са. Оценивание выполняется в режиме `model.eval()` под `torch.no_grad()` (листинг 2).

```
model.train()                    # Dropout is active only in training
for X_batch, y_batch in train_loader:
    optimizer.zero_grad()        # do not accumulate old gradients
    logits = model(X_batch.to(DEVICE))
    loss = criterion(logits, y_batch.to(DEVICE))   # logits, not softmax
    loss.backward()
    optimizer.step()
val_metrics, _, _, _ = evaluate_model(model, val_loader)  # eval + no_grad
if val_metrics["loss"] < best_val_loss - 1e-5:
    best_val_loss = val_metrics["loss"]
    best_state = copy.deepcopy(model.state_dict())
    epochs_without_improvement = 0
else:
    epochs_without_improvement += 1
```

Листинг 2: Эпоха обучения с ранней остановкой

Начальная конфигурация: Adam ($\eta = 10^{-3}$), 64 скрытых нейрона, без регуляризации; обучение остановилось на 96-й эпохе. На рисунке 4 Train loss снижается почти до нуля, а Validation loss выходит на плато около 0.17 — признак умеренного переобучения; validation macro-F1 стабилизируется около 0.957. Сравнение с baseline на validation приведено в таблице 4: MLP превосходит логистическую регрессию по macro-F1 на 0.0246, гипотеза о пользе нелинейности подтверждается.

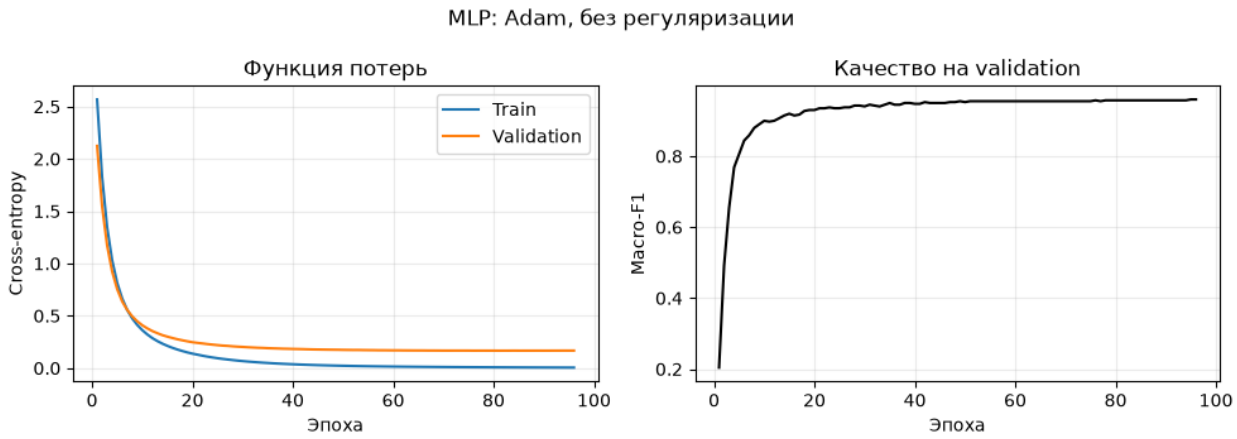


Рис. 4: Кривые обучения MLP (Adam, без регуляризации)

Модель	Accuracy	macro-F1	Cross-entropy	ROC-AUC
LogisticRegression	0.9325	0.9327	—	—
MLP (Adam)	0.9575	0.9573	0.1665	0.9980

Таблица 4: Validation-метрики начальной MLP и baseline

4.4 Сравнение оптимизаторов

SGD, Momentum и Adam сравнивались при одинаковых архитектуре, seed, разбиении и правиле остановки. Learning rate заданы заранее разных масштабов (таблица 5): для Adam — стандартный 10^{-3}, так как его шаг нормируется по градиенту; для SGD — 0.03; для Momentum ($\beta = 0.9$) — 0.02, поскольку инерция увеличивает эффективный шаг.

Оптимизатор	η	Эпох	best val loss	Val macro-F1
Adam	0.001	96	0.1665	0.9573
Momentum	0.020	43	0.1823	0.9525
SGD	0.030	120	0.1830	0.9474

Таблица 5: Сравнение оптимизаторов на validation

Лучший результат у Adam (macro-F1 = 0.9573), он выбран для дальнейших экспериментов. Momentum остановился на 43-й эпохе, SGD дошёл до предела в 120 эпох без срабатывания ранней остановки. Разница между Adam и SGD по macro-F1 составляет около 0.01; сравнение проведено на одном seed.

4.5 Сравнение Dropout и L2-регуляризации

Для Adam ($\eta = 10^{-3}$) сравнивались три режима; в каждом менялся только один механизм регуляризации (таблица 6).

Dropout дал наименьшую val loss (0.1385 против 0.1665 без регуляризации) и наибольший macro-F1 (0.9600); обучение до ранней остановки продолжалось дольше (109 эпох против 96), то есть переобучение наступает медленнее. L2 с $\lambda = 10^{-4}$ почти не изменил результат (0.9598 против

Режим	Dropout	weight_decay	Эпох	val_loss	Val macro-F1
Dropout	0.25	0	109	0.1385	0.9600
L2	0	10^{-4}	113	0.1645	0.9598
Без регуляризации	0	0	96	0.1665	0.9573

Таблица 6: Сравнение режимов регуляризации на validation

0.9573). Различия macro-F1 между режимами малы (не более 0.0027 при 400 объектах в Validation). По правилу выбора (максимум validation macro-F1) выбрана конфигурация: **Adam, $\eta = 10^{-3}$, 64 скрытых нейрона, Dropout 0.25, без weight decay**; число эпох для финального обучения — 94 (эпоха с минимальной val loss выбранного запуска).

4.6 Финальный test

Схема preprocessing заново обучена на Train+Validation (1600 объектов), выбранная MLP обучена 94 эпохи, baseline переобучен на тех же данных. Test использован один раз. MLP превзошла логистическую регрессию и по Accuracy, и по macro-F1 (таблица 7); cross-entropy MLP на test равна 0.0859. Лучшая линейная модель ЛР №1 имела на test macro-F1 = 0.9675, у MLP — 0.9824. Отчёт по классам — в таблице 8, нормированная confusion matrix — на рисунке 5.

Модель	Accuracy	Precision	Recall	F1-macro	ROC-AUC
LogisticRegression	0.9650	—	—	0.9649	—
MLP	0.9825	0.9831	0.9825	0.9824	0.9989

Таблица 7: Результаты на тестовой выборке

Слабее всего распознаётся класс 4 (recall = 0.90, 4 ошибки из 40); по одной ошибке допущено в классах 5, 9 и 10.

Класс	Precision	Recall	F1-score	Support
1	0.98	1.00	0.99	40
2	0.95	1.00	0.98	40
3	1.00	1.00	1.00	40
4	1.00	0.90	0.95	40
5	1.00	0.97	0.99	40
6	0.95	1.00	0.98	40
7	1.00	1.00	1.00	40
8	0.98	1.00	0.99	40
9	1.00	0.97	0.99	40
10	0.97	0.97	0.97	40
Accuracy		0.98		400
Macro avg	0.98	0.98	0.98	400
Weighted avg	0.98	0.98	0.98	400

Таблица 8: Отчёт о классификации (MLP, test)

4.7 Анализ ошибок

Из 400 тестовых объектов ошибочно классифицированы 7 (таблица 9); столбец Idx — позиция объекта в тестовой выборке. Для таблицы признаки выбраны по позициям `test_index` (в ноутбуке строки `raw_errors` идут в порядке возрастания индекса, а таблица ошибок отсортирована по уверенности, поэтому признаки выровнены заново).

Четыре из семи ошибок приходятся на класс 4 (объекты 231, 303, 369, 370), которые ушли в четыре разных класса (1, 2, 8, 6). **Возможные причины ошибок:**

1. **Перекрытие классов.** Объект 303 (`att1` = 5.36, `att2` = 5.74) лежит в области рисунка 2, где перемешаны классы 2, 3 и 4; близки к ней и ошибки 4→2 и 9→2. Класс 4 имеет широкий разброс по `att1`, поэтому пересекается с несколькими соседями.

2. **Нетипичные значения признаков.** У объекта 369 значение `att1` =

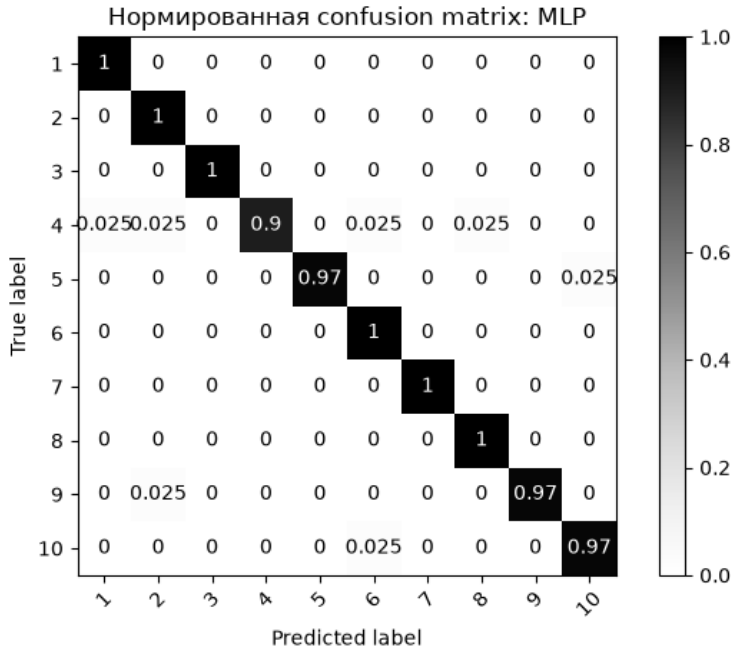


Рис. 5: Нормированная матрица ошибок (Confusion Matrix) для MLP

14.23 — выброс для класса 4 (рисунок 2, слева) и типичное значение для класса 8, в который он и отнесён. Объект 231 (`att1` $= -6.04$, `att2` $= -10.68$) лежит в области класса 1, в который он и отнесён с уверенностью 0.969.

3. **Возможный шум в данных и ограничения модели.** Объекты 385 и 231 классифицированы неверно с очень высокой уверенностью (0.979 и 0.969). Это может указывать на нестандартно написанные или неоднозначные цифры, а также на ограниченность модели с одним скрытым слоем, обученной на 1600 объектах.

Выводы

1. Линейный baseline (`LogisticRegression`) показал на validation Accuracy = 0.9325 и macro-F1 = 0.9327, на test после переобучения на Train+Validation — 0.9650 и 0.9649.

2. Ручной BackProp реализован корректно: максимальная относительная ошибка по 12 проверенным компонентам градиента равна $4.81 \cdot 10^{-10}$. Сеть 2–8–2 решила XOR (loss 0.0064 за 3000 эпох, 4 из 4 объектов

Idx	true	pred	conf	att1	att2	att4	att7
385	5	10	0.979	−3.19	−5.17	7.90	−3.15
231	4	1	0.969	−6.04	−10.68	6.01	−0.88
303	4	2	0.897	5.36	5.74	9.23	−1.71
308	10	6	0.680	1.84	−11.61	5.41	−2.42
369	4	8	0.679	14.23	5.82	2.31	−1.68
315	9	2	0.617	1.77	−3.32	7.01	−3.32
370	4	6	0.333	−4.37	4.43	7.71	8.81

Таблица 9: Ошибки MLP на test (по убыванию уверенности)

верно), что показывает роль скрытого слоя и нелинейности.

3. Начальная MLP (64 нейрона, Adam, $\eta = 10^{-3}$) на validation достигла macro-F1 = 0.9573 против 0.9327 у baseline, ROC-AUC = 0.9980; на кривых видно умеренное переобучение (Train loss $\to$ 0, Validation loss около 0.17).

4. Среди оптимизаторов лучшим оказался Adam (macro-F1 = 0.9573), затем Momentum (0.9525) и SGD (0.9474); сравнение проведено на одном seed.

5. Лучший режим регуляризации — Dropout 0.25 (val macro-F1 = 0.9600, val loss = 0.1385 против 0.1665 без регуляризации); L2 с $\lambda = 10^{-4}$ почти не повлиял (0.9598).

6. На test выбранная MLP показала Accuracy = 0.9825, macro-F1 = 0.9824, ROC-AUC = 0.9989 против 0.9650 и 0.9649 у логистической регрессии; лучшая линейная модель ЛР №1 дала macro-F1 = 0.9675.

7. Ошибки (7 из 400) сосредоточены в классе 4 (recall = 0.90, 4 ошибки); возможные причины — перекрытие классов, нетипичные значения признаков и возможный шум в данных.

8. Протокол соблюдён: preprocessing обучался только на Train (затем на Train+Validation), конфигурация выбрана по validation, test

использован один раз; оценка выполнялась в `model.eval()` под `torch.no_grad()`, градиенты обнулялись перед каждым мини-пакетом.